RL-Course 2025/26: Final Project Report

DRAI: Jannik Rombach, Adriano Polzer

February 26, 2026

1 Introduction

This work presents the development and evaluation of reinforcement learning agents based on the Twin Delayed Deep Deterministic Policy Gradient (TD3) and Soft Actor-Critic (SAC) algorithms for a competitive 1v1 hockey environment. The agents are implemented based on the original formulations of the respective algorithms as introduced in prior work, and are further extended with algorithmic and reward-based modifications to improve performance. Reinforcement learning in competitive environments is particularly challenging due to sparse reward structures and complex interaction dynamics between multiple agents. These characteristics complicate efficient exploration and stable policy optimization, motivating the use of additional training and algorithmic design strategies to improve learning efficiency and policy robustness.

To verify the correctness and robustness of the implementations, the algorithms are first evaluated in standard benchmark environments. Subsequently, the performance of the proposed modifications is analyzed in the competitive hockey environment, where the focus lies on improving tactical behavior and policy robustness. For training and evaluation, we developed a modular reinforcement learning framework based on PyTorch, enabling flexible configuration of environments, algorithmic variants, and reward modifications.

This report describes the methodology, including the implemented algorithms and improvements (Section 2), the evaluation, which is divided into experiments on simple benchmark environments and the hockey environment (Section 3), and a discussion of the results (Section 4).

2 Method

The hockey environment's fully continuous action space makes discrete-action methods such as Deep Q-Network (DQN) [7] inapplicable, and on-policy algorithms such as Proximal Policy Optimization (PPO) [9] less attractive due to poor sample efficiency. We therefore adopt two off-policy, continuous-control algorithms that share a common backbone yet represent complementary design philosophies: TD3 [2] optimizes a deterministic policy with explicit bias-correction mechanisms, while SAC [3] optimizes a stochastic policy through entropy maximization. Both maintain a replay buffer $\mathcal{D}$ of transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$, train an actor π and one or more critics Q_θ on minibatches from $\mathcal{D}$, and stabilize learning via slowly updated target networks through Polyak averaging,

$$\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta', \qquad \phi' \leftarrow \tau\,\phi + (1-\tau)\,\phi', \tag{1}$$

where $\tau \ll 1$ ensures the target parameters θ' and ϕ' track their online counterparts θ and ϕ slowly, providing a stable and consistent learning target throughout training. The individual algorithms are described in Sections 2.1 and 2.2, further improvements to both are presented in Section 2.3.

2.1 Twin Delayed Deep Deterministic Policy Gradient

TD3 [2] extends Deep Deterministic Policy Gradient (DDPG) [6] by adressing two fundamental failure modes of determenistic actor-critic methods: overestimation bias in the critic and instability in polciy updates. The actor encodes a *deterministic* policy $a = \pi_\phi(s)$ and is trained to maximize expected discounted return via the determenistic policy gradient theorem [11]:

$$\nabla_\phi J(\phi) \approx \mathbb{E}_{s\sim\mathcal{D}}\Big[\nabla_a Q_\theta(s,a)\big|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)\Big]\,. \tag{2}$$

In actor-critic methods, the policy is updated greedily with respect to the critic, meaning any overestimation in the value function is directly exploited by the actor, leading to suboptimal or diverging behaviour [2].
TD3 adresses this with three interacting mechanisms.

Clipped double Q-learning
Two independent critics Q_{θ_1} and Q_{θ_2} are trained in parallel but share no parameters. When computing Bellman targets, the minimum of the two target critics is used:

$$y = r + \gamma(1-d)\min_{i=1,2} Q_{\theta_i'}(s',\tilde{a}) \tag{3}$$

where d indicates episode termination and θ_i are the target network parameters updated via (1). Taking the minimum introduces a conservative bias that counteracts the systematic overastimation inherent in maximizing over noisy function approximators. Both critics are updated independently by minimizing the mean squarred Bellman error $(y - Q_{\theta_i}(s,a))^2$ over minibatches from D.
Since gradients flow independently through each critic, the two networks diverge slightly, preventing the correlated overestimation that arises when a single critic is used.

Target policy smoothing
To prevent the actor from exploiting narrow, misleading peaks in the value function, the target action is perturbed with clipped Gaussian noise before evaluating the Bellman target:

$$\tilde{a} = \mathrm{clip}\big(\pi_{\phi'}(s') + \epsilon,\ a_{\min},\ a_{\max}\big)\,, \quad \epsilon \sim \mathrm{clip}(\mathcal{N}(0,\sigma),\ -c,\ c)\,. \tag{4}$$

This encourages the critics to learn a smoother value landscape, making individual erroneous peaks less exploitable. The noise standard deviation σ and clip range c are hyperparameters that control the effective radius of this smoothing.

Delayed policy updates
Even with conservative targets and smoothed backups, updating the actor at every critic step introduces high variance into the policy gradient: the critic has not yet fully integrated the latest target information before its gradient is used to move the policy. TD3 therefore updates the actor every K critic steps, typically $K = 2$. This delay ensures the critics are in a more stable and accurate state when the actor gradient is computed, substantially reducing variance of policy updates and preventing the actor from over-adapting to transient critic errors [2]. Target networks for both actor and critics are updated via the shared Polyak rule (1), ensuring the learning targets change slowly and the training signal remains consistent across consecutive gradient steps.
Togehter, these three mechanisms directy counteract the core failure modes of deteremenistic actor-critic methods: clipped double Q-learning reduces overestimation bias, target policy smoothing regularizes the value function against sharp local peaks, and delayed updates reduce gradient variance during policy optimization.

2.2 Soft Actor-Critic

SAC is an off-policy actor-critic algorithm designed for optimizing *stochastic* policies in continuous action spaces. Our implementation follows the framework introduced by [3], which augments the standard maximum reward objective with an entropy maximization term [12]. This approach encourages wider exploration and improves robustness by seeking a policy that maximizes the trade-off between expected return and entropy:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}[r(\mathbf{s}_t,\mathbf{a}_t) + \alpha\,\mathcal{H}(\pi(\cdot|\mathbf{s}_t))] \tag{5}$$

where ρ_π denotes the state-action distribution induced by policy π and $\mathcal{H}$ represents the entropy of the policy at state $\mathbf{s}_t$.

The SAC algorithm consists of an actor, critic, and an entropy regularization term governed by a temperature parameter α. The actor is parameterized by a neural network that defines a Gaussian distribution over the continuous action space by estimating the mean $\mu_\theta(s)$ and standard deviation $\sigma_\theta(s)$ in its output layer.

To ensure differentiability of the sampling process with respect to the policy parameters, the reparameterization trick is applied [3, 5]. Instead of sampling actions directly from the Gaussian distribution, actions are expressed as $a = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$ is a standard normal random variable. The sampled actions are passed through a $\tanh$ function and scaled to match the action bounds of the environment. To correctly account for this squashing transformation in the entropy term, the log-probability is adjusted using the determinant of the Jacobian of the $\tanh$ transformation. The actor is trained on the maximum entropy objective [3]:

$$J(\pi_\phi) = \mathbb{E}_{(\mathbf{s}_t\sim\mathcal{D})}\Big[\mathbb{E}_{(\mathbf{a}_t)\sim\pi_\phi}[\alpha \log \pi_\phi(a_t|s_t) - Q_\theta(s_t,a_t)]\Big] \tag{6}$$

where for each state sampled from the replay buffer $\mathcal{D}$, an action is sampled from the actor and the critic estimates the value for each state-action pair.

As presented in the follow-up work [4], we employ two Q-networks for the critic instead of a single Q-network and a value network as originally proposed. The soft value function can be implicitly derived from the soft Q-function by combining the Q-values with the policy entropy term.

During training, two independent Q-networks estimate the value of a state-action pair. To mitigate overestimation bias, soft policy evaluation is performed using the minimum of the two Q-values, which provides a more conservative value estimate [2]. The critic is trained using a mean squared Bellman error loss:

$$J_Q(\theta_i) = \mathbb{E}_{\mathcal{D}}\Big[(Q_{\theta_i}(s_t,a_t) - y_t)^2\Big] \tag{7}$$

with target value

$$y_t = r_t + \gamma\left(\min_{j=1,2} Q_{\bar{\theta}_j}(s_{t+1},a_{t+1}) - \alpha \log \pi_\phi(a_{t+1}|s_{t+1})\right). \tag{8}$$

The target networks $\bar{\theta}_j$ are updated using soft updates with factor τ, which improves training stability by preventing rapid divergence between online and target critics.

Furthermore, automatic parameter tuning is implemented as introduced in the follow-up paper [4]. The temperature parameter α is optimized to match a target policy entropy, allowing the agent to automatically balance exploration and exploitation. The temperature is updated by minimizing

$$J(\alpha) = \mathbb{E}_{a_t\sim\pi_\phi}\left[-\alpha\left(\log \pi_\phi(a_t|s_t) + \bar{\mathcal{H}}\right)\right], \tag{9}$$

where $\bar{\mathcal{H}}$ denotes the target entropy, typically set to $-\dim(\mathcal{A})$ as recommended in [4]. The parameter α is optimized using a gradient-based method.

2.3 Improvements

We implemented a framework based on YAML configuration files to manage and persist training runs, enabling reproducible experiments and systematic tracking of hyperparameter changes. All modifications described below are applied on top of both the TD3 and SAC base implementations.

Replay Buffer

Since both implemented algorithms are off-policy, training relies on data stored in a replay buffer. We implemented standard Experience Replay (ER), which improves data efficiency by reusing past interactions and reducing correlations between subsequent samples. Transitions are stored as tuples (s_t, a_t, r_t, s_{t+1}) in a finite buffer and sampled uniformly during training. However, uniform sampling treats all transitions equally, ignoring that transitions with large Temporal Difference (TD) errors may provide stronger learning signal. To address this, we additionally implement Prioritized Experience Replay (PER) [8].
PER assigns higher sampling probabilities to transitions with larger TD errors. Each transition i receives priority $p_i = |\delta_i| + \varepsilon$, and is sampled according to $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, where α controls the degree of prioritization. The resulting bias is corrected via importance-sampling weights $w_i = (N^{-1} \cdot P(i)^{-1})^\beta$, with β annealed linearly to 1 over training. Efficient sampling and updates are implemented using a binary SumTree in $\mathcal{O}(\log N)$ time.

Pink Noise

Exploration in off-policy algorithms is commonly achieved by injecting noise into the actor's actions. Standard white noise $\mathcal{N}(0, \sigma)$ is temporally uncorrelated, leading to jittery exploration with no coherence across consecutive actions. Eberhard et al. proposed colored noise as a principled alternative, parameterized by a spectral exponent β [1]. It is generated by scaling the Fourier components of a white noise signal by $f^{-\beta/2}$, suppressing high frequencies to produce temporally correlated action perturbations. Pink noise ($\beta = 1$) was shown to best balance exploration and exploitation across continuous control tasks. We apply it as the exploration noise for TD3, replacing the standard Gaussian perturbation. SAC learns a stochastic policy with explicit entropy regularization and therefore does not require external exploration noise.

Reward Shaping

To enhance learning in the hockey environment, we extend the default reward formulation with additional shaping terms. The environment provides sparse rewards for winning and losing, with puck contact and velocity signals disabled by default. The total reward is given by $r = \sum_i w_i \cdot r_i$, where each r_i denotes a reward component and $w_i \in \mathbb{R}$ its corresponding weight.
In early training, the agent often required many timesteps before initiating puck contact. To accelerate this, we introduce a time-penalty r_{passive}, applied when the puck remains stationary on the agent's side beyond the first 10% of episode steps,

$$r_{\text{passive}}(t) = \begin{cases} -c & \text{if } x_{\text{puck}} \leq 0,\ \|v_{\text{puck}}\| < 0.1,\ \text{and } t > 0.1 \cdot T_{\max}, \\ 0 & \text{otherwise,} \end{cases} \tag{10}$$

where $c > 0$ is a configurable penalty coefficient.

To encourage defensive positioning, we additionally reward the agent for staying close to its own goal when the puck is on the opponent's side,

$$r_{\text{defense}} = \begin{cases} \dfrac{1}{1 + \|p_{\text{agent}} - g\|_2} & \text{if } x_{\text{puck}} > 0, \\ 0 & \text{otherwise,} \end{cases} \tag{11}$$

where g denotes the agent's goal center. All shaping components are individually configurable via the YAML configuration file.

Self-Play

To enable the agent to continuously improve against increasingly strong opposition, we implement a self-play training framework inspired by AlphaZero [10]. Rather than training against a fixed opponent, the agent faces a dynamically growing **opponent pool** that diversifies the training distribution and prevents overfitting to any single playing style.

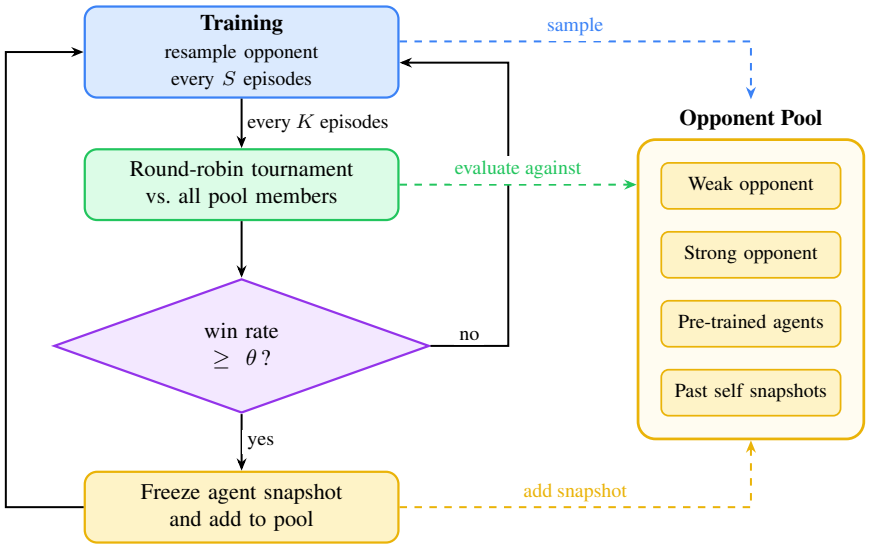

Figure 1: Self-play pool: Round-robin eval every K episodes vs. pool. Add snapshot if win rate $\geq \theta$.

The pool is initialized with a weak and a strong basic opponent and retains them permanently throughout training, serving as a minimum performance bar that the agent must reliably beat. Beyond these fixed baselines, the pool can also contain pre-trained agents of a different algorithm type, providing diverse and competent opposition.

During training, the agent's opponent is resampled every S episodes. Every K episodes, competence is assessed via a round-robin tournament where we evaluate against all pool members. The empirical win rate is

$$\hat{\rho} = \frac{W}{|\mathcal{P}| \cdot n_{\text{eval}}}, \tag{12}$$

where W is the number of wins, $|\mathcal{P}|$ the pool size, and n_{eval} the number of evaluation episodes per opponent. If $\hat{\rho} \geq \theta$, the agent's weights are frozen and we add snapshot to the pool, ensuring only genuinely stronger versions enter. To sample from the pool, a recency bias assigns higher probability to more recent snapshots, keeping the training challenge relevant while retaining older opponents for diversity. This creates a natural curriculum of continuously escalating opposition.

Table 1: Final Reward (Avg100 $\pm$ std)

	SAC + ER	SAC + PER	TD3 + ER	TD3 + PER
Pendulum	-145.96 $\pm$ 25.60	-144.72 $\pm$ 24.65	-147.11 $\pm$ 28.30	-150.81 $\pm$ 29.23
LunarLander	278.91 $\pm$ 15.18	261.56 $\pm$ 28.98	278.96 $\pm$ 12.55	273.12 $\pm$ 22.25
HalfCheetah	7580.42 $\pm$ 182.38	8908.11 $\pm$ 159.37	1726.27 $\pm$ 23.76	6421.28 $\pm$ 240.71

3 Evaluation

In this section, we evaluate the performance of the implemented algorithms and the proposed improvements across multiple experimental settings. The evaluation is structured in three stages. First, we assess the general performance and stability of the algorithms in three commonly used benchmark environments. Second, we analyze training behavior in the hockey environment, with particular focus on the impact of the introduced modifications, including self-play and other algorithmic adjustments. Finally, we evaluate the relative playing strength of the resulting agents in a competitive tournament setting. Match outcomes are aggregated and modeled using the Placket-Luce rating system in order to obtain a grounded ranking of agents. This allows a fundamental comparison of different training configurations that goes beyond the performance metrics of a single run.

3.1 Simple Environments

We evaluate both SAC and TD3 in three standard benchmark environments: Pendulum, Lunar Lander, and Half Cheetah. For each algorithm, we compare training with standard ER and with PER in order to assess the effect of prioritized sampling on learning dynamics and final performance. All algorithmic variants converge in Pendulum (200k steps) and LunarLander (800k steps), reaching comparable final returns. In these environments, PER does not lead to substantial performance differences.
In contrast, clearer distinctions emerge in HalfCheetah (800k steps). Both SAC and TD3 benefit substantially from prioritized replay, clearly outperforming their respective ER variants. Among all configurations, SAC with PER achieves the highest final results in this environment, outperforming the other approaches. The results further indicate that SAC generally converges faster than TD3 across environments, as reflected in steeper initial learning curves (see Appendix X). The hyperparameters used for all experiments are listed in Appendix Y.

3.2 Hockey Environment

After validating the algorithms on standard continuous control benchmarks, we evaluate them in the hockey environment. Each variant is trained for 2.5×10^6 timesteps against a mixture of the weak and strong basic opponent without self-play, serving as a first assessment of how the different improvements affect learning. Evaluation rewards are computed exclusively against the strong opponent using the sparse reward (-10 loss, 0 draw, $+10$ win). Figure 2 shows learning curves for a representative subset of tested configurations.
SAC converges reliably and quickly due to its automatic entropy regularization and effective off-policy data reuse. Adding reward shaping accelerates early learning, as penalizing passivity and rewarding puck contact guide the agent to engage with the puck sooner. However, aggressively weighted shaping rewards may partially overshadow the dominant win signal, causing the agent to prioritize intermediate behaviors over scoring, which manifests as a slightly delayed convergence. Based on these observations, we adopt mild reward shaping to preserve the primacy of the win signal.

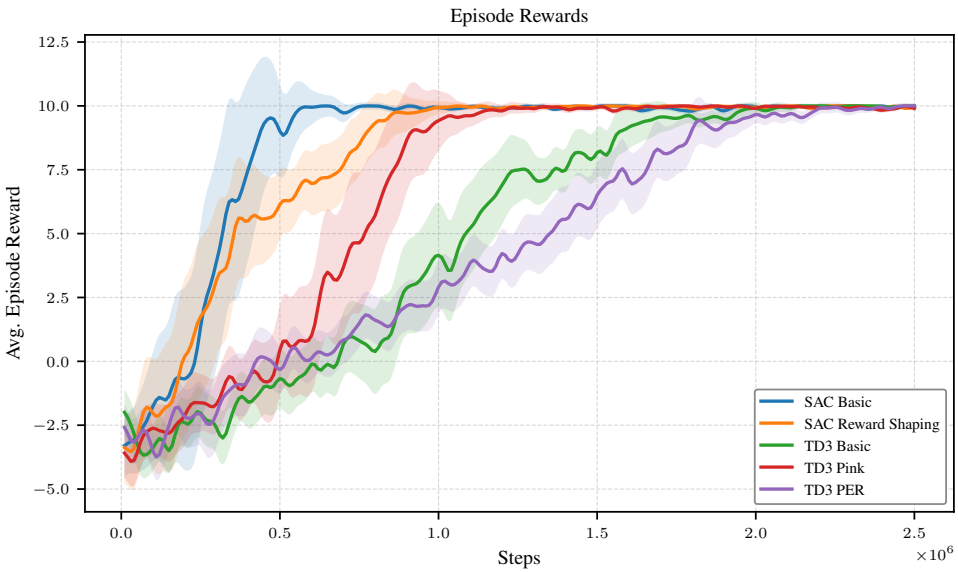

Figure 2: Avg. episode reward (mean ± eval variability).

TD3 with standard white noise converges slower than SAC, as the deterministic policy relies entirely on externally injected noise for exploration. Replacing white noise with pink noise yields a clear improvement: the temporal correlation encourages the agent to commit to exploratory directions over consecutive timesteps, producing more purposeful puck-approach behavior and faster convergence, consistent with the findings of Eberhard et al. [1].
Contrary to expectations, TD3 with PER converges slower than TD3 Basic. In early training, priorities change rapidly as the value function is still far from converged, and the sampling distribution may become dominated by a small set of transitions. We assume this instability, combined with the importance-sampling correction dampening updates during the β annealing phase, outweighs the benefit of focused sampling in this setting. All variants ultimately converge and reliably beat the strong opponent.
Additional training diagnostics including actor and critic losses as well as the SAC entropy temperature α are provided in Appendix B.1. The entropy temperature decays steadily over training, reflecting increasing policy confidence. The TD3 actor losses decrease monotonically into negative territory as expected, and critic losses remain bounded and gradually decrease across all variants, indicating stable value function learning.

Based on these experiments, both SAC and TD3 were configured with low-weight reward shaping and standard experience replay. For TD3, pink noise was selected as the exploration strategy given its faster convergence. In both cases, deeper networks with two to three hidden layers of moderate width outperformed shallower alternatives, suggesting the hockey environment benefits from additional representational capacity, while networks that were too deep or too wide introduced instability. These configurations serve as the baseline for subsequent self-play fine-tuning, with full hyperparameters listed in Tables 2 and 3 in the appendix.
Own vs. Standard -> trained gegen strong und weak
Plot max 4/ 5 models TD3 mit PER SAC (original vs own) TD3 (original vs own)
Self Play + Turnier -> Auswahl von model für Turnier
jeweils 1 selfplay (ohne reward shaping) jeweils 1 gegen strong/ weak trainiert jeweils reward shaping (mit oder ohne selfplay) jeweils 1 nur gegen strong trainiert basic strong/ weak

4 Discussion

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

References

[1] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, May 2023.

[2] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International Conference on Machine Learning (ICML)*, 2018.

[4] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *CoRR*, 2018.

[5] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *International Conference on Learning Representations (ICLR)*, 2014.

[6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[8] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. *arXiv: Learning*, 2015.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[10] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[11] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, pages 387–395. JMLR Workshop and Conference Proceedings, 2014.

[12] B. D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

A Simple Environment Diagnostics

Bla Bla Hier auch allgemein text und vlt. Hyperparameter

A.1 Training Curves

Bla Bla Bla Bla
Sui Sui Sui

A.1.1 Pendulum

Further training diagnostics for the Pendulum environment are shown in Figure 3. The SAC entropy temperature α decays steadily, reflecting increasing policy confidence. TD3 losses stabilize toward the end of training.

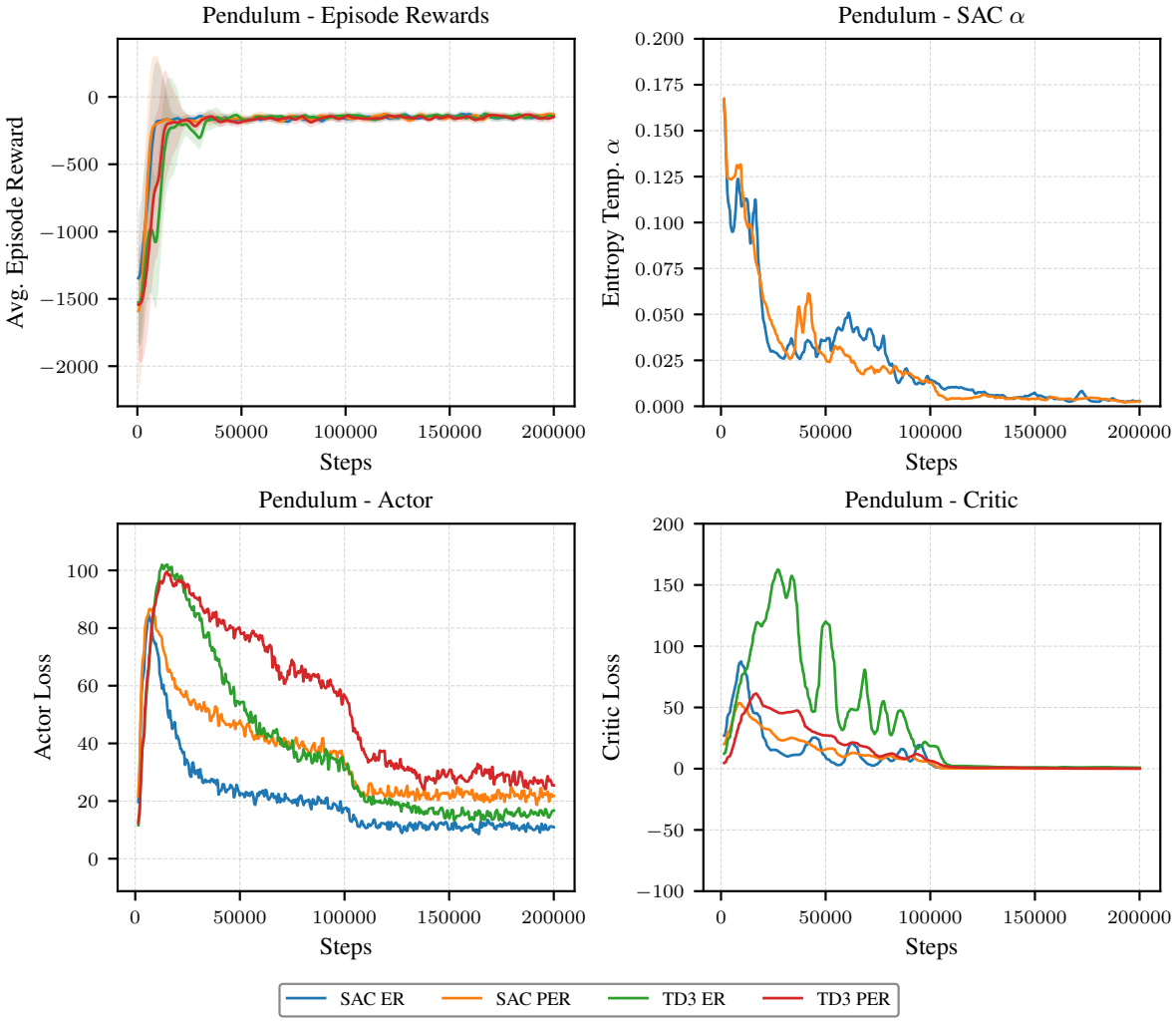

Figure 3: Training diagnostics for Pendulum environment. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

A.1.2 Lunar Lander

Training diagnostics for Lunar Lander are shown in Figure 4. Both algorithms show stable convergence, with TD3 exhibiting smoother critic losses.

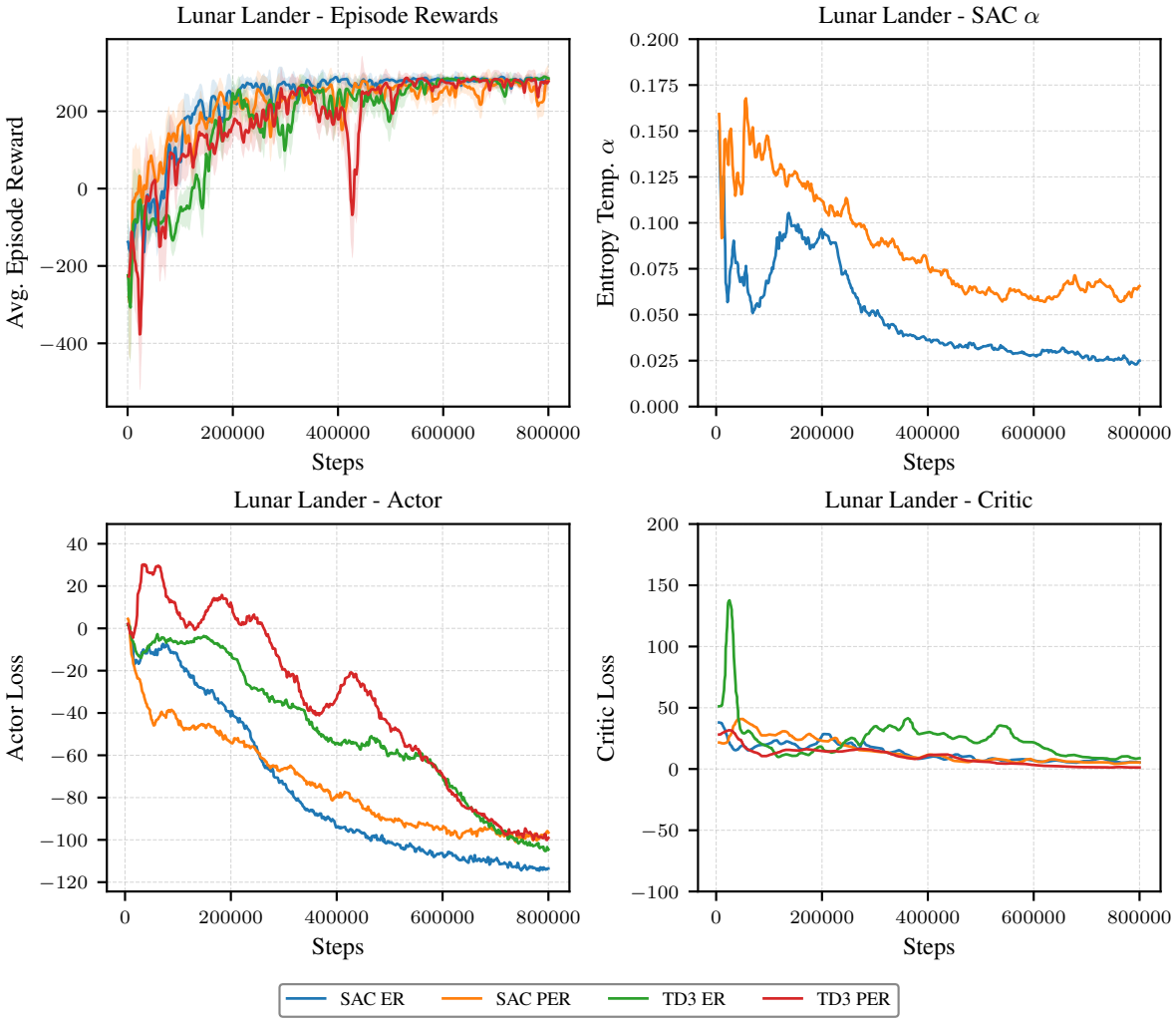

Figure 4: Training diagnostics for Lunar Lander environment. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

A.1.3 Half Cheetah

Half Cheetah diagnostics are shown in Figure 5. The higher-dimensional action space leads to slower convergence, but both variants achieve stable performance.

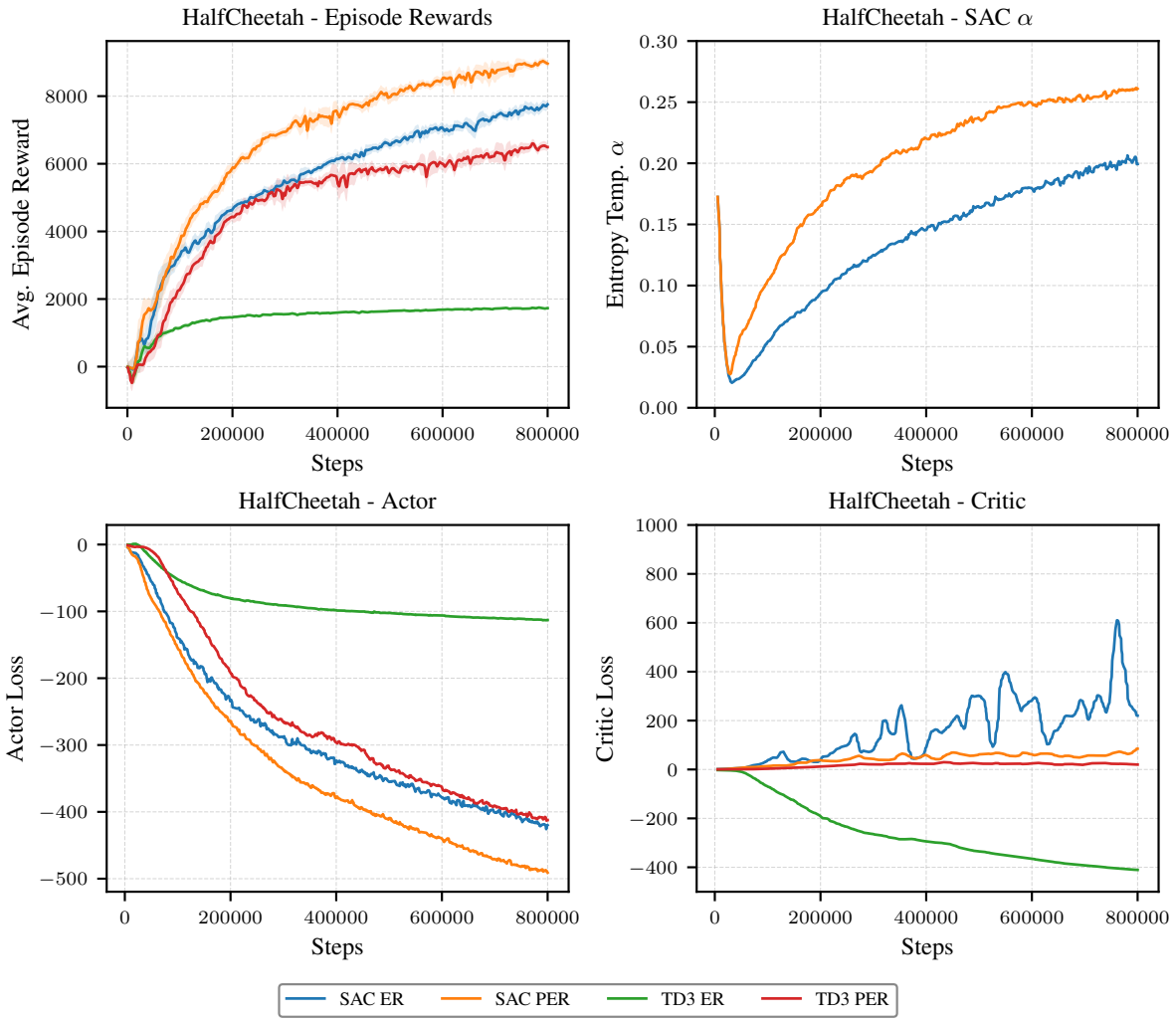

Figure 5: Training diagnostics for Half Cheetah environment. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

B Hockey Environment Diagnostics

sadshjashdjhaskjdhkjashdkjhaskjdhjasho ihchsaklhekhkjshdjkbawnbdwbdiuabkjbdw buiasijcbkjawbjhbjhbjchvbajwhbvjhd sadshjashdjhaskjdhkjashdkjhaskj dhjashoihchsaklhek hkjshdjkbawnbdwbdiuabkjbd buiasijcbkjawbjhbjhbjchvbajwhbvjhd

B.1 Training Curves

Further training diagnostics are shown in Figure 6. The SAC entropy temperature α decays steadily for both variants, reflecting increasing policy confidence. The TD3 actor losses decrease monotonically into negative territory as the actor learns to maximize the critic's Q-values. The critic loss of TD3 Basic appears noisier than the other variants, which may be attributable to the uncorrelated white noise producing more inconsistent transition patterns in the replay buffer. Critic losses decrease across all variants towards the end of training, indicating stable convergence.

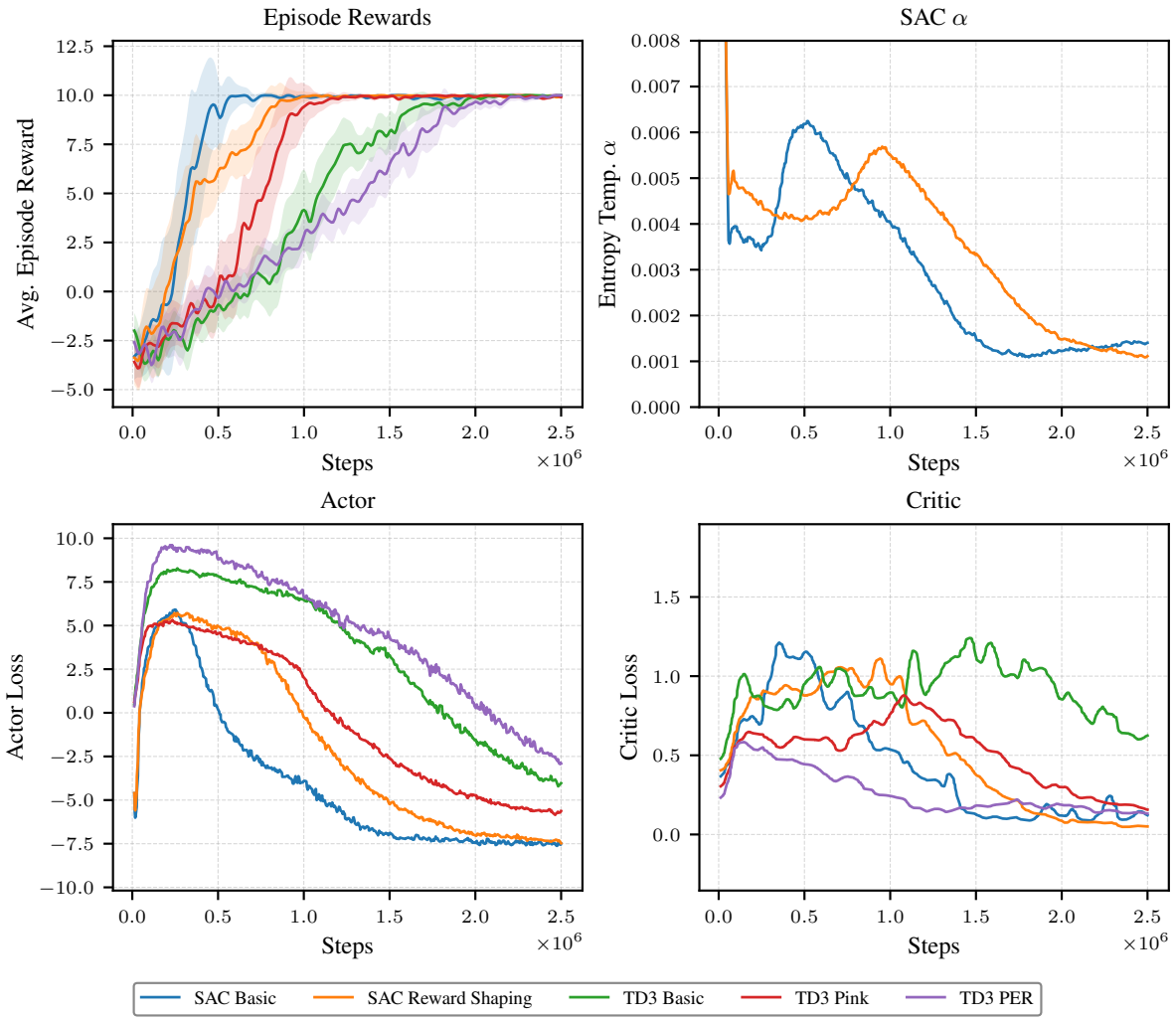

Figure 6: Training diagnostics for selected hockey environment configurations. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

B.2 Hyperparameters

Table 2: SAC hyperparameters.

Hyperparameter	**Value**
Discount factor γ	0.98
Soft update τ	0.005
Actor learning rate	3×10^{-4}
Critic learning rate	3×10^{-4}
α learning rate	1×10^{-4}
Initial entropy coefficient α	0.2
Target entropy	$-\lvert\mathcal{A}\rvert$
Actor architecture	$[512, 256]$
Critic architecture	$[512, 256, 128]$
Replay buffer size	10^6
Batch size	256
Learning starts	$10{,}000$
Reward shaping: touch puck	0.1
Reward shaping: puck direction	0.05

Table 3: TD3 hyperparameters.

Hyperparameter	**Value**
Discount factor γ	0.98
Soft update τ	0.005
Actor learning rate	3×10^{-4}
Critic learning rate	3×10^{-4}
Policy noise σ	0.2
Noise clip	0.5
Policy delay	2
Exploration noise (pink, $\beta = 1$)	0.15
Actor architecture	$[256, 256, 256]$
Critic architecture	$[256, 256, 128]$
Replay buffer size	10^6
Batch size	256
Learning starts	$10{,}000$
Reward shaping: touch puck	0.1
Reward shaping: puck direction	0.05

List of Acronyms

DQN Deep Q-Network

PPO Proximal Policy Optimization

DDPG Deep Deterministic Policy Gradient

TD3 Twin Delayed Deep Deterministic Policy Gradient

SAC Soft Actor-Critic

ER Experience Replay

PER Prioritized Experience Replay

TD Temporal Difference